

CSD_04 — Mobile Applications & Services + Web Technologies

Paper-I, Unit 6 — "Mobile Application & Services" and Paper-II, Unit 5 — "Web Technologies", taken together in one file because they are really one story: building a piece of software that a user touches, whether it lives on a phone or in a browser.

Official syllabus wording, Paper-I §6: *"Factors in Developing Mobile Applications, Frameworks and Tools in Mobile Applications, Text-to-Speech Techniques, Designing the Right UI, Multichannel and Multimodal UIs, Storing and Retrieving Data, Synchronization and Replication of Mobile Data, Android Storing and Retrieving Data, Putting It All Together: Packaging and Deploying."*

Official syllabus wording, Paper-II §5: *"The Phases of web site development: Implementation, Maintenance, Testing. Basic HTML Concepts, HTML, HEAD, TITLE, BODY, Paragraphs, Lists, Formatted and Unformatted Text, Hyperlink, Font (Size, Colour), image. PHP: Server-side web scripting, Installing PHP, Adding PHP to HTML, Syntax and Variables, Passing information between pages, Basic PHP error/problems."*

Networking mechanics (OSI, TCP/IP, IP addressing) are covered in **CORE_05_ComputerNetworks.md** — this file only covers the client-server HTTP cycle as it applies to a website request, not network layering. Nothing here overlaps **CORE_09_ProgrammingC_CPP.md** either — PHP is a different language, taught from scratch below.

Why this matters / where it appears

1. **Both halves are guaranteed, standalone, low-competition units.** Nothing else in the CS Degree syllabus touches mobile development or PHP/HTML — this is 100% new content, which means it is *entirely* yours to control, unlike, say, OS scheduling which nine other candidates also know cold.
2. **The 2025 papers hit both halves hard, in prose-answer form, not MCQ trivia.** Paper-I carried a 15-mark "frameworks and tools in mobile app development" question and a 15-mark "designing the right UI for mobile" question — that is 30 of 100 Paper-I marks from this one unit alone. Paper-II carried a 10-mark "GET vs POST" question, a 10-mark "PHP syntax and variables" question, and a 15-mark "phases of website development" question.
3. **The examiner wants explanatory prose, not bullet trivia.** Look at the 2025 answer style: full paragraphs that reason from constraints to conclusions ("the screen is small, the input is a fingertip, so touch targets must be at least 48×48dp, so..."). MCQ-style fact lists lose marks here. Practice **writing**, not just reading, the paragraphs below.
4. **PHP/HTML code must actually run in your head.** Every code block below has its output traced by hand — do the same for anything you write in the exam. A `$_GET` vs `$_POST` mix-up is a guaranteed trap question.

Topic checklist

Tick a box only when you can write the paragraph or trace the code from a blank page.

§	Topic	Depth	Done
1	Factors/constraints in mobile app development	15-mark, asked 2025	- []
2	Native vs hybrid vs web apps; frameworks and tools	15-mark, asked 2025	- []

Input method	Touch (imprecise, no hover, multi-touch), not mouse+keyboard	Larger touch targets; gestures instead of right-click menus
Processing power & memory	Weaker CPU, less RAM, no swap space of consequence	Lightweight data structures; avoid memory leaks (OS kills the app under pressure)
Fragmentation	Many OS versions, many manufacturers, many screen shapes in the wild simultaneously	Test across a device/OS matrix; use compatibility libraries
Interruptions	Calls, notifications, screen lock, app switch, at any moment	The app must save state and resume gracefully — this is <i>the</i> reason for the Android activity lifecycle (§7)
Context of use	Outdoors in glare, one-handed, walking, distracted	Short tasks, high contrast, forgiving touch targets
App store review & distribution	Gatekept release channel, versioned rollout	Build, sign and package correctly before every release (§9)
Security	Device can be lost, stolen, rooted; app runs on hardware the developer does not control	Encrypt local data; do not trust client-side validation alone
Multiple platforms	iOS, Android, sometimes web, side by side	Choose native / hybrid / cross-platform trade-off (§2) up front — expensive to change later

Likely exam questions

1. Discuss the various factors that must be considered when developing a mobile application. **[15]** (2025 pattern)
2. Why does mobile application design differ fundamentally from desktop software design? **[10]**
3. Explain how battery and network constraints influence mobile app architecture. **[5]**

MCQ traps

- "Fragmentation" in mobile computing means device/OS diversity, **not** file/memory fragmentation (that is an OS-unit term — do not confuse across units).
- Density-independent units are **dp** (Android) / **pt** (iOS), not raw pixels.

2. Native vs hybrid vs web apps; frameworks and tools

Concept — in plain English first

There are three ways to put an app on somebody's phone, and the difference is simply: **which language do you write in, and does the OS know your code exists as a "real" app or not.**

Key points — the three approaches

	Native	Hybrid	Web / PWA

Language	Platform's own (Kotlin/Java on Android, Swift/Obj-C on iOS)	Cross-platform framework compiling/bridging to native (React Native, Flutter)	HTML/CSS/JavaScript, runs in a browser or wrapped in one (Ionic + Capacitor)
Performance	Best — direct OS/hardware access	Good — near-native (Flutter compiles to ARM) to good (RN bridges to native widgets)	Weakest — runs inside a browser engine
Code reuse across platforms	None — separate codebase per OS	High — one codebase, most logic shared	Highest — one codebase, no app-store build at all
Access to device hardware (camera, sensors, contacts)	Full , immediate on every OS release	Good, via plugins; may lag behind new OS APIs	Limited, browser-API dependent
Distribution	App store only	App store (packaged)	App store optional; can be a plain URL
Look and feel	Matches OS conventions automatically	Matches if using platform-styled widgets	Must be hand-matched; often feels "web-like"
Development cost	Highest (2x team/skillset for 2 platforms)	Medium	Lowest
Example frameworks	Android SDK + Android Studio, Xcode + Swift	React Native (JavaScript, bridges to native), Flutter (Dart, compiles to native ARM code, own rendering engine, strong hot reload)	Ionic + Capacitor (web app in HTML/CSS/JS wrapped in a native shell)

Frameworks and tools to be able to name (this is what the 2025 15-marker rewarded):

Category	Tools
IDEs	Android Studio (official Android IDE, Gradle build system), Xcode (iOS)
Cross-platform frameworks	React Native, Flutter, Ionic/Capacitor, Xamarin
Local storage	SQLite, Room (Android's SQLite abstraction), Core Data (iOS), Realm, key-value stores
UI guideline systems	Material Design (Android), Human Interface Guidelines (iOS)
Testing	JUnit and Espresso (Android), XCTest (iOS), Appium and Detox (cross-platform end-to-end)
Debugging/monitoring	Android Studio Profiler, Xcode Instruments, Flipper, Sentry/Crashlytics (crash reporting)

Choosing between them — the one-paragraph answer: native wins when performance or full hardware access is critical (games, camera-heavy apps); hybrid/cross-platform wins when time-to-market and one shared codebase across iOS+Android matter more than squeezing out the last bit of performance; a web app/PWA wins when you need zero-install reach and app-store approval is undesirable or too slow.

Likely exam questions

1. Discuss the different frameworks and tools commonly used in mobile application development. **[15]**
(*exactly the 2025 question*)
2. Differentiate between native, hybrid and web-based mobile applications, with examples of each. **[10]**

3. Compare React Native and Flutter as cross-platform frameworks. [5]

MCQ traps

- Flutter compiles to **native ARM code**; React Native **bridges** JavaScript to native widgets — these are different mechanisms, commonly swapped in MCQs.
- Ionic/Capacitor apps are fundamentally **web apps** wrapped in a native shell, not compiled native code.

3. Text-to-speech techniques

Concept — in plain English first

Text-to-speech (TTS) converts written text into spoken audio, so the app can be used hands-free/eyes-free (driving, accessibility for the visually impaired, notifications read aloud).

Key points

Stage	What happens
Text analysis / normalisation	Expand abbreviations, numbers, dates into full words ("Dr." → "Doctor", "12/5" → "the twelfth of May")
Linguistic analysis	Determine pronunciation, word stress, sentence intonation (prosody)
Waveform synthesis	Two main techniques:
— Concatenative synthesis	Stitches together pre-recorded snippets of real human speech; natural-sounding but limited to recorded vocabulary/voice
— Parametric / formant synthesis	Generates speech from a mathematical model of the vocal tract; more flexible (any text, any speed/pitch), can sound more robotic
— Neural TTS (modern)	Deep learning models (e.g. WaveNet-style) generate waveforms directly; most natural-sounding, used in modern assistants
Platform APIs	Android: <code>TextToSpeech</code> class; iOS: <code>AVSpeechSynthesizer</code> ; Web: <code>SpeechSynthesis</code> Web API

Likely exam questions

1. Explain the working of text-to-speech systems in mobile applications. [5]
2. Differentiate between concatenative and parametric speech synthesis. [5]

MCQ traps

- TTS is **text** → **speech**; speech recognition (STT / ASR) is the reverse — do not swap them.

4. Designing the right UI for mobile

Concept — in plain English first

Mobile UI design is the discipline of working *within* the constraints from §1, not fighting them. Every rule below traces back to one of: small screen, imprecise touch input, interruption-prone usage, or platform convention.

Key points

Constraint	Design rule that follows
Screen fragmentation (thousands of sizes/densities across Android; notches and safe areas on iOS)	Use responsive/constraint-based layout and density-independent units (dp/pt), not fixed pixels
Fingertip imprecision	Minimum touch target 48×48 dp (Android) / 44×44 pt (iOS) , adequate spacing between targets
Platform convention	Follow Material Design (Android) or Human Interface Guidelines (iOS) — users expect platform-native navigation patterns (back button behaviour, tab bars)
Interrupted, one-handed, outdoors, in-motion usage	Short tasks; large legible text; high contrast; avoid tasks that need two hands or sustained attention
Limited screen real estate	Progressive disclosure (show only what is needed now); avoid dense desktop-style menus
Users get lost easily on small screens	Clear visual hierarchy; the next action should be obvious, reachable (thumb zone) and reversible (undo, back)

The one-line thesis to open the essay with: *"Mobile UI design succeeds when the interface makes the next action obvious, reachable and reversible — every specific rule (touch target size, platform convention, short tasks) is a consequence of that one goal under the constraints of a small screen and an imprecise, interruptible way of interacting with it."*

Likely exam questions

1. Discuss the challenges and considerations involved in designing the right UI for mobile applications. **[15]** (*exactly the 2025 question*)
2. What is the minimum recommended touch target size, and why? **[5]**
3. Compare Material Design and Human Interface Guidelines as UI convention systems. **[5]**

MCQ traps

- Minimum touch target: **48×48 dp** Android, **44×44 pt** iOS — these numbers, and which unit belongs to which OS, are MCQ-tested.

5. Multichannel and multimodal UIs

Concept — in plain English first

Multichannel = the same service delivered over more than one *channel* (app, SMS, web, voice IVR, in-branch kiosk) — the user picks the channel; think of a bank offering mobile app, website and phone-banking IVR for the same account.

Multimodal = a *single* interaction that supports more than one input/output *modality*

simultaneously — voice + touch + gesture in the same interface, e.g. saying "call mom" while also being able to tap a contact.

	Multichannel	Multimodal
Axis of variation	Channel (delivery medium)	Modality (interaction mode)
Example	Same banking service via app, SMS, web, IVR	Voice assistant that also accepts touch and gesture in one session
Design challenge	Consistency of data/state across channels	Fusing/disambiguating simultaneous inputs

Likely exam questions

1. Distinguish between multichannel and multimodal user interfaces, with examples. [10]
2. Why is consistency across channels important in a multichannel system? [5]

MCQ traps

- "Multichannel" and "multimodal" are the single most confused pair of terms in this unit — channel = *where*, modality = *how*.

6. Storing and retrieving data; synchronisation and replication

Concept — in plain English first

A mobile device is frequently offline, so an app cannot simply talk to a central server for every read/write the way a web app can. It needs **local storage** for when offline, and a **sync strategy** to reconcile local changes with the server once connectivity returns.

Key points

Local storage options (platform-agnostic):

Option	Best for
Key-value store (SharedPreferences/UserDefaults)	Small settings, flags
Embedded relational DB (SQLite / Room / Core Data)	Structured, queryable data
Object/NoSQL store (Realm)	Object-graph data, offline-first apps
Flat files	Media, large blobs, exported data

Synchronisation and replication:

Concept	Meaning
Synchronization	Keeping the local copy and the server copy of data consistent over time
Replication	Maintaining copies of the same data in multiple places (device + server, or across servers) so each side can operate independently and be reconciled later

Conflict	Arises when the same record is changed both locally (offline) and on the server before sync
Conflict resolution strategies	Last-write-wins (timestamp), server-always-wins, client-always-wins, manual merge/user prompt, operational transform / CRDTs for advanced collaborative apps
Sync triggers	On app foreground, on network reconnect, periodic background sync, push-triggered sync
Offline-first design	Treat the local store as the source of truth for the UI; sync happens in the background and never blocks the interface

Likely exam questions

1. Explain the concepts of synchronization and replication of mobile data. **[10]**
2. What conflict-resolution strategies exist when the same record is edited both offline and on the server? **[5]**
3. Compare the local data storage options available to a mobile application. **[10]**

MCQ traps

- Synchronization $\neq$ replication: sync is about *keeping consistent over time*; replication is about *having multiple copies at all*.
- "Offline-first" means the **local store is authoritative for the UI**, not that the app ignores the server.

7. Android: architecture stack, activity lifecycle, four components, intents

Concept — in plain English first

Android is the concrete worked example the syllabus wants for "storing and retrieving data" and for the architecture behind everything above. Learn its stack top-down and its lifecycle as a state machine — that is the shape every exam question here takes.

7.1 Android architecture stack

```

+-----+
| Applications          (the apps users install) |
+-----+
| Application Framework (Activity Manager, Content |
|                       Providers, View System,   |
|                       Notification Manager, etc.) |
+-----+
| Libraries + Android Runtime (ART) |
|   Native C/C++ libs (SQLite, WebKit, SSL, media) |
|   ART executes bytecode (successor to Dalvik VM) |
+-----+
| Hardware Abstraction Layer (HAL) |
+-----+
| Linux Kernel          (drivers, power mgmt, |
|                       memory mgmt, process mgmt) |
+-----+

```

Draw this as five stacked horizontal bands, labelled bottom-to-top: **Linux Kernel** → **HAL** →

7.2 The four Android application components

Component	Role
Activity	One screen with a UI the user interacts with
Service	Runs in the background, no UI (e.g. playing music, downloading)
Broadcast Receiver	Responds to system-wide or app broadcast events (e.g. battery low, SMS received)
Content Provider	Manages a structured, shared set of app data, exposing it to other apps via a standard interface

7.3 The activity lifecycle — the state machine to draw



Callback	Fired when	Typical use
<code>onCreate()</code>	Activity is first created	One-time setup: inflate layout, bind views
<code>onStart()</code>	Activity becomes visible	
<code>onResume()</code>	Activity gains foreground focus, user can interact	Start animations, camera preview, sensors
<code>onPause()</code>	Another activity partially/fully covers this one	Release resources shared with other apps quickly (camera), save lightweight state
<code>onStop()</code>	Activity no longer visible at all	Release heavier resources
<code>onRestart()</code>	Stopped activity is coming back to the foreground	
<code>onDestroy()</code>	Activity is finishing, or OS is reclaiming memory	Final cleanup

Why the lifecycle exists at all — the answer that gains marks: it exists *because* of the mobile constraint from §1 — interruptions (calls, notifications, app switching) can end an app's

foreground time at any moment, and the device has limited memory so the OS may kill backgrounded processes without warning. The lifecycle gives the app defined callback points to save state before that happens, and to restore it cleanly when the user returns.

7.4 Intents

An **Intent** is a messaging object used to request an action from another app component.

Type	Meaning	Example
Explicit intent	Names the exact target component (by class)	Launching a specific activity within your own app
Implicit intent	Declares a general action; the OS resolves which app/component can handle it	"Share this text" → OS offers all apps that registered to handle sharing

Intents also carry **extras** (key-value data payload) and can request a result back (`startActivityForResult`, or the modern Activity Result API).

7.5 Android storage options (the syllabus explicitly names this: "Android Storing and Retrieving Data")

Option	What it stores	Persistence scope
SharedPreferences	Small key-value pairs (settings, flags)	App-private, survives app restarts
Internal storage	Files private to the app	Deleted when app is uninstalled
External storage	Files on shared/removable storage	May be readable by other apps (with permission)
SQLite database (via Room)	Structured relational data	App-private, persists until uninstall or explicit clear
Network / cloud storage	Remote data, fetched and cached	Not local; requires sync strategy (§6)

7.6 The Android Manifest

`AndroidManifest.xml` is the app's declaration file, required in every Android project. It declares:

- All four components used by the app (activities, services, receivers, providers) — components not declared here cannot run.
- The app's **permissions** (camera, internet, location, ...) requested from the user/OS.
- The **minimum and target SDK versions**.
- The app's **package name**, version code/name.
- Hardware/software features required (e.g. camera, GPS).
- The **launcher activity** (which activity opens when the user taps the app icon), marked with an intent-filter for `MAIN/LAUNCHER`.

Likely exam questions

1. Describe the Android architecture stack with the help of a diagram. **[10]**
2. Explain the Android activity lifecycle with a diagram. Why is it necessary? **[10]**

3. What are the four fundamental components of an Android application? Explain each briefly. [10]
4. Differentiate between explicit and implicit intents, with an example of each. [5]
5. What is the role of the Android Manifest file? List four things it declares. [5]
6. Describe Android's data storage options and when each is appropriate. [10]

MCQ traps

- The four Android components: **Activity, Service, Broadcast Receiver, Content Provider** — a fifth fake option ("Intent Filter", "Fragment") is a common distractor; Fragment is a UI sub-component, not one of the four.
- `onPause()` vs `onStop()`: pause = partially/fully covered but potentially visible; stop = fully invisible. Order is always Pause → Stop, never reversed.
- An implicit intent may match **zero, one, or many** apps; the OS shows a chooser if more than one.
- The Manifest file is `AndroidManifest.xml`, not `.json` or `.properties`.

8. Packaging and deploying: APK, signing, Play Store

Concept — in plain English first

Writing the code is only half the job — "Putting It All Together" (the syllabus's own phrase) is the pipeline that turns source code into something installable and distributable.

Key points

Step	What happens
1. Build/compile	Source (Kotlin/Java) + resources compiled into <code>.dex</code> bytecode
2. Package into APK	The APK (Android Package) bundles compiled code, resources, assets and the manifest into one archive; AAB (Android App Bundle) is the modern Play-Store-preferred format that lets Google Play generate optimised APKs per device
3. Sign	Every APK/AAB must be digitally signed with a developer's private key before it can be installed on a device or accepted by a store. Signing proves the app's identity and that updates come from the same author — Android refuses to install an update signed with a different key than the original
4. Test build variants	Debug build (unsigned or debug-signed, for development) vs release build (signed with the production key, optimised/minified)
5. Versioning	<code>versionCode</code> (integer, must increase for every Play Store update) and <code>versionName</code> (human-readable, e.g. "2.3.1")
6. Upload & listing	Upload the signed AAB/APK to the Play Console; provide store listing (title, description, screenshots, privacy policy, content rating)
7. Review	Google Play reviews the app for policy compliance before publishing
8. Staged rollout	Release to a percentage of users first, monitor crash reports, then expand

9. Updates

New versions must be re-signed with the **same** key and a higher `versionCode`

Why signing matters enough to be its own exam point: it is the mechanism that lets the OS and the store trust that an "update" to an app genuinely comes from the original developer and has not been tampered with — losing your signing key means you can never again publish an update to the same app listing.

Likely exam questions

1. Describe the process of packaging and deploying an Android application, from build to Play Store. **[10]**
2. Why must an APK be digitally signed? What happens if a signing key is lost? **[5]**
3. Differentiate between `versionCode` and `versionName`. **[5]**

MCQ traps

- APK = **Android Package**, the installable archive; AAB = **Android App Bundle**, the upload format Play Store now prefers.
- An app **cannot** be installed/updated on a device without a valid signature.
- `versionCode` must strictly **increase** on every Play Store submission; `versionName` is just a display string with no such constraint.

PART B — WEB TECHNOLOGIES

9. Phases of website development

Concept — in plain English first

Building a website has a lifecycle shaped like the general SDLC (`CORE_07_SoftwareEngineering.md`) but with web-specific emphases the syllabus explicitly names: **Implementation, Maintenance, Testing** — plus the planning/design work that necessarily precedes them.

Key points

Phase	What happens
1. Planning / requirements	Define purpose, audience, content scope, budget, timeline
2. Design	Wireframes, visual design, information architecture; design responsively for mobile/tablet/desktop widths; design interactive states (hover, active, disabled)
3. Implementation	Front end: mark up pages semantically in HTML, style with CSS, add behaviour with JavaScript. Back end: build server logic and data storage using a stack such as PHP + MySQL, Node.js, Django or Laravel
4. Testing	Functional testing (links, forms work), cross-browser/cross-device testing, performance testing, security testing (e.g. against SQL injection, XSS)
5. Deployment	Deploy to a live web server / hosting provider, configure domain and DNS
6. Maintenance	Ongoing: content updates, security patches, bug fixes, monitoring uptime

The essential difference from general software development, worth stating in the answer: *a website is never "finished" — maintenance and iteration are continuous*, unlike a shrink-wrapped desktop application with discrete release cycles. Content changes, security patches, and browser/device drift all force ongoing work after "launch."

A commonly under-budgeted item, worth naming: cross-browser/cross-device testing — different browsers render subtly differently, and this is a very common cause of last-minute delay in a website project.

Likely exam questions

1. Describe the phases of website development, from planning to maintenance. **[15]** (*exactly the 2025 question*)
2. How does website development differ from general software development in terms of its lifecycle? **[5]**
3. What testing considerations are specific to websites, as opposed to desktop software? **[10]**

MCQ traps

- The three phases the syllabus explicitly names are **Implementation, Maintenance, Testing** — if a question asks "which three phases does the syllabus emphasise", these three (not "planning" or "design") are the literal answer.

10. HTML in detail

Concept — in plain English first

HTML is a **markup language**, not a programming language — it describes the *structure and meaning* of content (this is a heading, this is a list, this is a link), and the browser decides how to render that structure. Every tag has a **start tag**, optional **content**, and an **end tag**: `<tag>content</tag>`. Some tags are **void elements** with no content/end tag (`<img>`, `<br>`, `<hr>`).

10.1 Basic document structure

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph of unformatted text.</p>
</body>
</html>
```

Tag	Role
<code><!DOCTYPE html></code>	Declares HTML5 to the browser
<code><html></code>	Root element, wraps the whole document

<code><head></code>	Metadata: not rendered directly in the page body
<code><title></code>	Text shown in the browser tab
<code><body></code>	Everything the user actually sees
<code><h1></code> – <code><h6></code>	Headings, decreasing importance
<code><p></code>	Paragraph

10.2 Formatted vs unformatted text

Tag	Effect
<code></code> / <code></code>	Bold (strong also carries semantic emphasis)
<code><i></code> / <code></code>	Italic (em also carries semantic emphasis)
<code><u></code>	Underline
<code>
</code>	Line break (void element)
<code><pre></code>	Preformatted text — preserves whitespace/line breaks exactly as typed
<code><sub></code> / <code><sup></code>	Subscript / superscript

10.3 Lists

```

<ul>                                <!-- unordered list -->
  <li>Tea</li>
  <li>Coffee</li>
</ul>

<ol>                                <!-- ordered list -->
  <li>Wake up</li>
  <li>Study</li>
</ol>

<dl>                                <!-- definition list -->
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
</dl>

```

10.4 Hyperlinks

```

<a href="https://example.com">Visit Example</a>
<a href="page2.html">Go to Page 2</a>           <!-- relative link -->
<a href="mailto:test@example.com">Email us</a>
<a href="#section1">Jump within this page</a>   <!-- anchor link -->

```

`href` = hyperlink reference (destination). Opening in a new tab: `target="_blank"`.

10.5 Font (size, colour)

```
<font size="5" color="blue">Old-style font tag (deprecated in HTML5)</font>
<p style="font-size:20px; color:blue;">Modern way: inline CSS</p>
```

Exam point: `<font>` is deprecated in HTML5; the correct modern approach is CSS (`font-size`, `color` properties) — but the syllabus explicitly lists "Font (Size, Colour)" so know both the legacy tag and the modern replacement.

10.6 Images

```

```

`src` = source path/URL (required). `alt` = alternate text, shown if the image fails to load and read by screen readers (accessibility). `width/height` in pixels.

10.7 Tables

```
<table border="1">
  <tr>
    <th>Name</th><th>Marks</th>
  </tr>
  <tr>
    <td>Asha</td><td>85</td>
  </tr>
  <tr>
    <td>Toshi</td><td>90</td>
  </tr>
</table>
```

Tag	Role
<code><table></code>	Wraps the whole table
<code><tr></code>	Table row
<code><th></code>	Header cell (bold, centred by default)
<code><td></code>	Data cell
<code>colspan</code> / <code>rowspan</code> (attributes on <code><td></code> / <code><th></code>)	Merge cells across columns/rows

10.8 Forms

```
<form action="submit.php" method="post">
  Name: <input type="text" name="uname"><br>
  Password: <input type="password" name="pwd"><br>
  Gender:
  <input type="radio" name="gender" value="M">Male
  <input type="radio" name="gender" value="F">Female<br>
  Subjects:
  <input type="checkbox" name="subj[]" value="Maths">Maths
  <input type="checkbox" name="subj[]" value="Sci">Science<br>
  Country:
  <select name="country">
    <option value="IN">India</option>
```

```
<option value="US">USA</option>
</select><br>
<textarea name="comments" rows="4" cols="30"></textarea><br>
<input type="submit" value="Submit">
</form>
```

action = URL the form data is sent to. **method** = **get** or **post** (see §14 — this is the exact 2025 exam pairing of concepts: HTML forms in this section, GET vs POST in the PHP section). Every input needing to reach the server **must have a name attribute** — inputs without **name** are never submitted.

Likely exam questions

1. Explain the basic structure of an HTML document with a diagram/example. [5]
2. Write HTML to create an ordered list, an unordered list and a definition list. [5]
3. Explain hyperlinks in HTML with examples of absolute, relative and anchor links. [5]
4. Write HTML to create a table with a header row and merged cells (colspan/rowspan). [10]
5. Write an HTML form containing a text box, radio buttons, a checkbox, a dropdown and a submit button. Explain the role of the **action** and **method** attributes. [10]
6. Differentiate between the **** tag and CSS for controlling text appearance. [5]

MCQ traps

- ****, **
, **<hr>, **<input>** are **void elements** — no closing tag.
- An **<a>** tag's destination attribute is **href**, **not src** (that is for ****).
- Form controls without a **name** attribute are **not sent** to the server on submit.
- **<th>** is bold/centred by **default rendering**, not because of any required CSS.

11. CSS basics

Concept — in plain English first

If HTML is the skeleton (structure/meaning), **CSS (Cascading Style Sheets) is the skin** — it controls colour, layout, spacing and typography, separately from the content.

Key points

Three ways to apply CSS:

```
<!-- 1. Inline -->
<p style="color:red;">Inline styled text</p>

<!-- 2. Internal (in <head>) -->
<style>
  p { color: red; font-size: 16px; }
</style>

<!-- 3. External (best practice) -->
<link rel="stylesheet" href="style.css">
```

Selectors:

Selector	Matches
<code>p</code>	Every <code><p></code> element
<code>.classname</code>	Every element with <code>class="classname"</code>
<code>#idname</code>	The single element with <code>id="idname"</code>
<code>p.warning</code>	A <code><p></code> that also has class <code>warning</code>

The cascade / specificity, in increasing priority: browser default → external/internal stylesheet (by selector specificity: element < class < id) → inline style → `!important`.

The box model — every HTML element is a box: `content` → `padding` → `border` → `margin` (innermost to outermost). This is the single most-asked CSS diagram — draw four nested rectangles labelled margin (outermost), border, padding, content (innermost).

Likely exam questions

1. What is CSS? Explain the three ways of applying CSS to an HTML page. [5]
2. Explain the CSS box model with a diagram. [5]
3. Differentiate between class and id selectors in CSS. [5]

MCQ traps

- `id` selectors should be **unique per page**; `class` selectors can repeat.
- Inline style has **higher** priority than an external/internal stylesheet rule of the same specificity.

12. PHP: server-side scripting

Concept — in plain English first

Two categories of "web scripting" exist, and the syllabus wants you to know the difference before anything else:

	Client-side	Server-side
Runs where	In the user's browser	On the web server, before the page is sent
Example languages	JavaScript	PHP , Python (Django), Node.js, Java (Servlets)
Can access the database directly?	No (would expose credentials)	Yes
Output the browser receives	The script itself, plus its effect on the page	Only the resulting HTML — the PHP source is never sent to the browser
Typical use	Form validation, animation, interactivity without a round trip	Generating dynamic HTML, database access, authentication, session/cookie logic

The one sentence that defines PHP: *PHP is a server-side interpreted scripting language — the server executes the PHP code and sends only the resulting HTML output to the browser; the client never sees the PHP source.*

12.1 Installing PHP / adding PHP to HTML

- Install a PHP runtime (XAMPP/WAMP bundle PHP + Apache + MySQL together for local development, or install PHP standalone alongside a web server).
- The file extension **must be** `.php` — the web server hands `.php` files to the PHP interpreter; a `.html` file is served as-is and any PHP inside it is ignored (sent to the browser as literal text).
- PHP is embedded directly inside HTML using the tags:

```
<!DOCTYPE html>
<html>
<body>
<h2>Marks Report</h2>
<?php
    echo "Hello, World!";
?>
</body>
</html>
```

Anything **outside** `<?php ... ?>` is sent to the browser unchanged; anything inside is executed on the server. In a file that is pure PHP, the closing `?>` is conventionally **omitted** to avoid accidentally emitting trailing whitespace/newlines to the browser.

12.2 Syntax, variables, types

```
<?php
    $name = "Asha";           // variable: always starts with $, no type declaration
    needed
    $marks = 85;              // integer
    $percentage = 85.5;       // float
    $passed = true;           // boolean
    $subject = null;          // null

    echo "Name: " . $name . ", Marks: " . $marks; // '.' concatenates strings
?>
```

Output: `Name: Asha, Marks: 85`

Rule	Detail
Variable naming	Must start with <code>\$</code> , then a letter or underscore, then letters/digits/underscores
Typing	Dynamically typed — no declaration, type is inferred from the assigned value, and can change at runtime
Case sensitivity	Variable names are case-sensitive (<code>\$Name</code> $\neq$ <code>\$name</code>); function/keyword names are not
Comments	<code>//</code> or <code>#</code> single line, <code>/* ... */</code> multi-line
Variable variables	<code>\$\$a</code> refers to the variable whose name is the <i>value</i> of <code>\$a</code> — e.g. if <code>\$a = "marks"</code> , then <code>\$\$a</code> is the same as <code>\$marks</code>

Data types: integer, float (double), string, boolean, array, object, NULL, resource.

Operators:

Category	Examples
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> (exponentiation)
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> , and <code>===</code> (identical — value and type)
Logical	<code>&&</code> <code>&</code> <code> </code> <code> </code> <code>!</code> (also <code>and</code> , <code>or</code>)
String	<code>.</code> (concatenation), <code>.=</code> (concatenate-assign)
Increment/decrement	<code>++</code> <code>--</code>

`==` vs `===` — a classic PHP MCQ: `"5" == 5` is **true** (loose comparison, type-juggled);
`"5" === 5` is **false** (strict — string vs integer are different types).

12.3 Control structures

```
<?php
$marks = 72;
if ($marks >= 90) {
    echo "Grade A";
} elseif ($marks >= 60) {
    echo "Grade B";
} else {
    echo "Grade C";
}
?>
```

Output: Grade B

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i . " ";
}
?>
```

Output: 1 2 3 4 5

Also available: `while`, `do...while`, `switch`, `foreach` (for arrays), `break`, `continue`.

12.4 Functions

```
<?php
function average($a, $b) {
    return ($a + $b) / 2;
}
echo average(80, 90);
?>
```

Output: 85

PHP also supports default parameter values, variable-length argument lists, and pass-by-reference (`function inc(&$x) { $x++; }`).

12.5 Arrays

```
<?php
    $subjects = array("Maths", "Science", "English");    // indexed array
    echo $subjects[1];                                // Science

    $marks = array("Maths" => 85, "Science" => 90);      // associative array
    echo $marks["Science"];                            // 90

    foreach ($marks as $subject => $mark) {
        echo "$subject: $mark\n";
    }
?>
```

Output:

```
Science
90
Maths: 85
Science: 90
```

Likely exam questions

1. Differentiate between client-side and server-side scripting, with examples. **[5]**
2. What are the fundamental syntax rules and the use of variables in PHP scripting? **[10]** (*exactly the 2025 question*)
3. Explain PHP data types, operators and control structures with examples. **[10]**
4. Write a PHP script to find and display the largest of three numbers. **[10]**
5. Explain indexed and associative arrays in PHP with an example of each. **[5]**
6. Trace the output of a given PHP script (loops/conditionals). **[5/10]**

MCQ traps

- PHP files **must** use the `.php` extension for the server to invoke the interpreter.
 - `==` is loose (type-juggling) comparison; `===` requires **same value and same type**.
 - PHP variables need **no** type declaration and **no** semicolon-terminated type keyword — only `$name = value;`.
 - The closing `?>` is optional and conventionally **omitted** in pure-PHP files.
-

13. Passing information between pages: GET vs POST, sessions, cookies, form handling

Concept — in plain English first

HTTP itself is **stateless** — the server has no memory of who you are between one request and the next. Everything in this section exists to work around that one fact: GET/POST are ways to *send* data on a single request; sessions/cookies are ways to make the server *remember* you across many requests.

13.1 GET vs POST

```
<form action="process.php" method="get">
  Name: <input type="text" name="uname">
  <input type="submit">
</form>
```

Submitting "Asha" sends the browser to:
`process.php?uname=Asha` — data appended visibly to the URL as a query string.

```
<?php
  echo "Hello, " . $_GET["uname"];
?>
```

Output (in the browser): `Hello, Asha`

```
<form action="process.php" method="post">
  Password: <input type="password" name="pwd">
  <input type="submit">
</form>
```

```
<?php
  echo "Received password of length: " . strlen($_POST["pwd"]);
?>
```

Here the data travels in the **HTTP request body**, not the URL — it never appears in the address bar.

	GET	POST
Where data travels	Appended to the URL as a query string	In the HTTP request body
PHP superglobal	<code>\$_GET</code>	<code>\$_POST</code>
Visible to the user / bookmarkable	Yes	No
Data size limit	Limited (URL length, ~2048 chars in practice)	Effectively unlimited (server-config dependent)
Cached / stored in browser history	Yes	No
Suitable for	Search queries, filtering, bookmarkable links	Login forms, passwords, file uploads, sensitive data
Idempotent / safe to repeat	Yes (should not change server state)	No (may create/modify data — should not be silently repeated)

13.2 Sessions and cookies

	Cookie	Session
Where stored	Client's browser	Server (a session ID is stored client-side in a cookie or URL)

Typical use	Remember-me tokens, preferences, tracking	Login state, shopping cart, temporary per-visit data
Lifetime	Set explicitly (can persist across browser restarts)	Ends when the browser closes or the session times out, unless made persistent
Security	Visible/editable by the user; must not store sensitive data raw	More secure — the actual data stays server-side

```
<?php
    session_start();                // must be the very first thing PHP outputs
    $_SESSION["username"] = "Asha"; // store in the session
    echo $_SESSION["username"];     // retrieve it on this or a later request
?>
```

Output: Asha

```
<?php
    setcookie("username", "Asha", time() + 3600); // expires in 1 hour
    echo $_COOKIE["username"] ?? "not set";       // on a later page load: Asha
?>
```

Common error: `session_start()` must be called **before any HTML/output is sent** — calling it after `echo` or any whitespace before `<?php` produces the classic *"headers already sent"* error.

13.3 Form handling — the complete example

```
<!-- login.html -->
<form action="login.php" method="post">
    Username: <input type="text" name="user"><br>
    Password: <input type="password" name="pass"><br>
    <input type="submit" value="Login">
</form>
```

```
<?php
    // login.php
    $user = $_POST["user"] ?? "";
    $pass = $_POST["pass"] ?? "";
    if ($user == "admin" && $pass == "1234") {
        session_start();
        $_SESSION["user"] = $user;
        echo "Login successful";
    } else {
        echo "Invalid credentials";
    }
?>
```

Output (correct credentials): Login successful

Likely exam questions

1. Explain the difference between the GET and POST methods in HTML forms. **[10]** (*exactly the 2025 question*)
2. Differentiate between sessions and cookies in PHP, with examples. **[10]**

3. Write a PHP script that receives form data and validates a login. [10]
4. Why is HTTP called "stateless", and how do sessions/cookies solve that problem? [5]

MCQ traps

- Cookies are stored **client-side**; sessions are stored **server-side** (with only an ID cookie on the client) — this exact pair is the most common MCQ in this topic.
- GET data is visible in the URL and browser history — **never** put a password in a GET form.
- `session_start()` must be called before any output — a very common "basic PHP error" the syllabus explicitly names.

14. PHP + MySQL connection and common errors

Concept — in plain English first

A dynamic website's real value is reading/writing a database on every request. PHP connects to MySQL through the `mysqli` extension (or PDO); the pattern is always **connect** → **query** → **process result** → **close**.

Key points

```
<?php
    $conn = mysqli_connect("localhost", "root", "", "school_db");
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }

    $result = mysqli_query($conn, "SELECT name, marks FROM students");

    while ($row = mysqli_fetch_assoc($result)) {
        echo $row["name"] . " scored " . $row["marks"] . "<br>";
    }

    mysqli_close($conn);
?>
```

Output (for two rows in the table):

```
Asha scored 85
Toshi scored 90
```

The four `mysqli_connect()` arguments, in order: host, username, password, database name.

Common PHP errors and problems (the syllabus explicitly names this)

Error	Typical cause
Parse error: syntax error, unexpected ...	Missing semicolon, unmatched brace/quote

Undefined variable / undefined array key	Using <code>\$_GET/\$_POST</code> key that was not actually submitted; always check with <code>isset()</code>
Headers already sent	Output (even whitespace) sent before <code>session_start()</code> / <code>setcookie()</code> / <code>header()</code>
Connection failed	Wrong host/username/password/database name, or MySQL service not running
Call to undefined function	Extension (e.g. <code>mysqli</code>) not enabled in <code>php.ini</code> , or a typo in the function name
SQL injection vulnerability	Concatenating raw <code>\$_POST/\$_GET</code> values directly into a SQL query string instead of using prepared statements

A line worth writing in any "common errors" answer: always validate/sanitise input and prefer **prepared statements** (`mysqli_prepare` / PDO bound parameters) over string-concatenated queries — this is the standard defence against SQL injection, and ties this section back to database security (`CORE_06_DBMS.md`).

Likely exam questions

1. Write a PHP script to connect to a MySQL database and display all records from a table. **[10]**
2. List and explain common errors encountered in PHP scripting. **[5]**
3. How does SQL injection occur in a poorly written PHP script, and how is it prevented? **[5]**

MCQ traps

- `mysqli_connect()` argument order: **host, username, password, database** — a shuffled-order MCQ is common.
- "Headers already sent" is caused by output **before** `session_start()/header()`, not after.

15. Client-server HTTP request-response cycle

Concept — in plain English first

Every website interaction, underneath the HTML/PHP above, is one instance of this cycle repeated: the browser (client) asks for something, the server answers.

Key points

1. User types a URL / clicks a link / submits a form
2. Browser resolves the domain name to an IP address (DNS lookup)
3. Browser opens a TCP connection to the server (port 80/443)
4. Browser sends an HTTP request:
 - Request line: `GET /page.php HTTP/1.1` (or `POST`)
 - Headers: `Host`, `Cookie`, `Content-Type`, ...
 - Body: (form data, if `POST`)
5. Server receives the request, routes it (e.g. to the PHP interpreter for a `.php` file)
6. Server-side script executes (queries DB if needed), builds an HTML response
7. Server sends an HTTP response:
 - Status line: `HTTP/1.1 200 OK` (or `404`, `500`, `302`, ...)

Headers: Content-Type, Set-Cookie, ...

Body: the HTML page

8. Browser parses and renders the HTML/CSS/JS, and the connection may close or be reused (keep-alive)

Draw this as a simple two-box diagram — Client on the left, Server on the right, one arrow labelled "HTTP Request" going right, one arrow labelled "HTTP Response" going left, with the DNS lookup shown as a small side-box the client consults before the request arrow.

Common status codes	Meaning
200 OK	Success
301 / 302	Redirect (permanent / temporary)
404 Not Found	Resource does not exist
500 Internal Server Error	Server-side script/error

Likely exam questions

1. Describe the client-server request-response cycle for a web page. [5]
2. What happens between a user clicking a link and the page appearing in the browser? List the steps. [10]

MCQ traps

- DNS resolution happens **before** the HTTP request is sent, not as part of it.
- 404 = resource not found (client-side path error); 500 = server-side script failure — these are frequently swapped in MCQs.

Master cross-reference — where each syllabus phrase lives in this file

Syllabus phrase	Section
Factors in Developing Mobile Applications	§1
Frameworks and Tools in Mobile Applications	§2
Text-to-Speech Techniques	§3
Designing the Right UI	§4
Multichannel and Multimodal UIs	§5
Storing and Retrieving Data / Sync and Replication	§6
Android Storing and Retrieving Data	§7.5
Packaging and Deploying	§8
Phases of web site development (Implementation, Maintenance, Testing)	§9

Basic HTML Concepts / HEAD / TITLE / BODY / Paragraphs / Lists / Text / Hyperlink / Font / Image	§10
PHP: Server-side scripting, Installing PHP, Adding PHP to HTML, Syntax and Variables	§12
Passing information between pages, Basic PHP error/problems	§13, §14